Implementing BUBBLE-BLUE over BLE L2CAP: An Open Mesh with Private Groups

Paul-Maxence Baraton, Nadjib Achir, Philippe Jacquet
INRIA, France
{paul-maxence.baraton, nadjib.achir, philippe.jacquet}@inria.fr

Wilfred Doré
Qualcomm, France
wdore@qti.qualcomm.com

***Abstract*—BUBBLE-BLUE (BB) is a multi-hop private group that rides on an *open* Bluetooth mesh: any BB node may forward, only holders of a group key may read. Prior work specified the construction and evaluated Connected Dominating Set (CDS) election in simulation. This paper reports the first implementation, on BLE L2CAP Connection-Oriented Channels, without Bluetooth pairing. We describe the on-wire format, the split between a topological plane and an application plane, and a repeatable evaluation on two Linux single-board computers built on Qualcomm SoCs: the Thundercomm RUBIK Pi 3 (QCS6490) and the Arduino UNO Q (QRB2210). A three-node mesh completes in 7–14 s, remains complete for 97–98% of a steady run, and delivers every chat message in automated runs; a six-node manual soak delivered 427 of 429, losing only the first and the last. Spread through a building on battery, six boards formed a four-hop line of about 60 m on their own; the far end received 76 of 79 messages at a median delay of 162 ms. Driving the shipped forwarder in simulation, the smallest TTL that reaches every node equals the source's hop eccentricity: the CDS adds no path stretch. The practical limit is therefore a hop budget: in simulation (unit-disk radio, nominal degree 6), the default TTL 10 spans about twenty nodes; this figure is not a hardware measurement. AES-256-GCM runs at 30k–173k packets/s, far above the BLE link; the BlueZ stack of the UNO Q saturates near 10 messages/s. Revocation excludes the victim in every trial and strands legitimate members only inside the join window, before public keys have flooded.**

***Index Terms*—Bluetooth Low Energy, L2CAP, mesh networks, connected dominating set, group communication.**

I. Introduction

Infrastructure-less local communication still collapses, in practice, onto a Wi-Fi access point or a smartphone OS that will not host a third-party mesh. Bluetooth is ubiquitous and needs no operator. The SIG Mesh profile [1], however, is a different stack, designed for managed flooding (lighting, sensors), not for a private group that forms between people who share a key and nothing else. The settings we have in mind are those where infrastructure is absent, saturated or distrusted: a crowd at an event, a site without coverage, an emergency response.

BUBBLE-BLUE [2] proposed the opposite split: an *open topological mesh* that any BB node may join and relay, and *private application bubbles* readable only with a group key. Routing is a dynamic CDS. That note gives the algorithms and unit-disk simulations. It does not give a BLE transport, a frame layout, a revocation protocol that reaches two-hop members, or measurements on hardware.

This paper fills that gap. The implementation runs over BLE L2CAP CoC [3] on two single-board computers: the Thundercomm RUBIK Pi 3, built on the Qualcomm QCS6490, and the Arduino UNO Q, which pairs a Qualcomm Dragonwing QRB2210 with an STM32U585 microcontroller. Connections use `BT_SECURITY_LOW`: there is no SMP pairing, no bond, and no address allow-list. Membership is the group key. A node that has never met another can still talk to it. A passive sniffer in range can read routing headers; it cannot read payloads (Section III-E).

Our contributions are:

1) A complete BB stack on BLE CoC, including the 26-byte routing header, packet types, opaque forwarding, and epoch-based revocation (Section III).
2) A repeatable multi-board evaluation method and the figures it produces on two platforms (Section V).
3) Characterised operating limits: hop-TTL, join-window revocation, and per-platform saturation—including two implementation defects the limit studies exposed (a shared anti-replay window and a split-brain coordinator election) and that are now fixed (Section V-D).

The rest of this paper is organised as follows. Section II positions BB against CDS literature and existing BLE messengers. Section III presents the design, its threat model, and the link discipline the platforms impose. Section IV describes the six-board demonstration stand. Section V evaluates the implementation, in simulation, on the bench and on the stand. Section VI concludes.

II. Related work

a) CDS and multi-hop broadcast.: Wu and Li's 1999 local rule [4] is our default election; their 2001 scheme [5] is the broader dominating-set routing setting. Multipoint relays [6] are the OLSR construction [7]; in BB, MPR is an alternative *algorithm* whose *result* is still a CDS. Jacquet's analytic densities [8] and the 1D/2D unit-disk comparison in [2] are why Wu–Li was preferred for a first implementation: simpler, and better than MPR on the 2D disk in that note. Dumbo II [9] is the emergency-network precedent cited in [2]. BLE has no native IP-style broadcast: every local flood is a sequence of L2CAP unicasts. That cost model is why a CDS (fewer transmitters) matters more here than on Wi-Fi MANETs.

TABLE I
WHAT BB IS NOT. "OPEN RELAY" MEANS A NODE MAY FORWARD A FRAME IT CANNOT DECRYPT.

System	Data plane	Backbone	Open relay
SIG Mesh [1]	managed flood	provisioned	no
BitChat [10]	GATT	none	n/a
Briar [11]	RFCOMM, Wi-Fi, Tor	contacts	no
Reticulum [12]	multi-transport	identity	no
BUBBLE-BLUE	LE CoC	dynamic CDS	yes

b) BLE transports.: We use LE Connection-Oriented Channels [3], not RFCOMM (Classic, serial, no BLE equivalent) and not GATT as the data plane (serialised writes, no native fan-out to CDS neighbours). Advertising carries *presence* only. SIG Mesh [1] remains a separate, provisioned stack.

c) Adjacent messengers.: BitChat [10] shows that a BLE-only chat can exist at consumer scale; it floods over GATT and elects no backbone. Briar [11] is delay-tolerant sync between *pre-paired* contacts over Classic RFCOMM, Wi-Fi and Tor—a different problem. Reticulum [12] is the closest Linux BLE stack; it is identity-routed, not CDS-routed. Bridgefy [13], [14] and BLEMeshChat [15] likewise flood or store-and-forward without a CDS; the analysis of Bridgefy in [14] is also a reminder of what a BLE mesh used in a crowd must withstand. None of these is a BB node.

Table I summarises the distinction. SIG Mesh is a product profile with its own provisioning. BitChat shows that a phone can chat over BLE alone, by flooding GATT writes. Briar synchronises mailboxes between people who have already paired. Reticulum routes to cryptographic identities across whatever link is up. BB is the remaining cell: a CDS elected on an open BLE mesh, with confidentiality only inside a group that shares a key. A node can sit in the CDS and relay for a bubble it does not belong to. None of the other four rows does that.

III. DESIGN

A BB node is a Linux process that owns an HCI adapter, accepts CoC connections on PSM `0x0081`, and runs the control and forwarding logic below. Pairing is unused. Confidentiality is application-layer.

A. Open mesh, private bubble

Two planes share the same links and the same 26-byte header, and nothing else.

The **topological plane** is open. Any BB node may advertise presence, accept a connection, take part in CDS election, and relay a frame it cannot decrypt. AES-GCM failure is logged; it is never a reason to drop the relay.

The **application plane** is private. Group traffic, key announcement and revocation are readable only with the current group key. Membership is that 32-byte secret, not a Bluetooth bond.

This is the construction of [2], realised as opaque forwarding of group data by nodes that may not belong to the bubble.

TABLE II
PLATFORMS AND SOFTWARE STACKS UNDER TEST.

	RUBIK Pi 3 (Thundercomm)	Arduino UNO Q
SoC	Qualcomm QCS6490	Qualcomm QRB2210 + STM32U585 MCU
BT controller	Broadcom BCM4345C5	Qualcomm WCN399x
HCI version*	9 (Core 5.0)	10 (Core 5.1)
HCI driver	`hci_uart_bcm`	`hci_uart_qca`
OS image	Ubuntu 24.04.3	Debian 13
Kernel	6.8.0-1055-qcom	7.0.0 (vendor)
BlueZ	5.72	5.82

*As reported by the controller (`btmgmt info`) on the boards used for every run in this paper.

B. Presence and sparse links

A node advertises a compact presence record in a connectable legacy advertisement (manufacturer-specific data under company identifier `0xFFFF`, followed by the magic `BB`). `0xFFFF` is the value the Bluetooth SIG reserves for internal and interoperability testing; its use here marks the prototype as experimental, and a deployed product would need an assigned identifier. The record carries an anonymous identifier, CDS flag, and the BD_ADDR at which it accepts CoC. Scanners accumulate a *presence catalog* indexed by anon_id. The catalog survives a dropped socket and is how a neighbour is redialled without pairing.

Links are not a full mesh. Each node caps its L2CAP degree and prefers CDS successors. On a six-node soak, boards held two or three links each and still reached the other five. When the CDS is empty—at start-up, and whenever legacy advertisements cannot carry neighbour lists under a $\sim$27-byte budget—a node dials from the catalog rather than waiting for an air-only election that cannot see two-hop topology.

Table II lists the two software stacks under test. Every platform limit reported below is a property of that stack (kernel, driver, BlueZ release, OS image) as we ran it, not of the silicon alone. The daemon requires CPython 3.14 (3.12 and 3.13 ship on the images): 3.14 is the first release whose `socket` module exposes the LE address types (`BDADDR_LE_PUBLIC`, `BDADDR_LE_RANDOM`) and accepts the four-field L2CAP address $(bdaddr, psm, cid, bdaddr_type)$. Earlier releases can only bind or connect BR/EDR L2CAP, so an LE CoC cannot be opened from the standard library.

A privileged daemon owns the adapter; the UI is an unprivileged client on an `AF_UNIX` socket, so restarting a display does not drop links. The same split is what lets a panel be a spectator: it draws peers, hops and revocation state, and it never holds a CoC. Figure 1 shows the layers of one node.

The compact advertisement is 23 bytes of manufacturer value before any neighbour list: magic and version (4), anonymous identifier (8), sequence (4), flags (1), reachable BD_ADDR (6). Legacy advertising accepts about 27 bytes of manufacturer data in total, so the neighbour and MPR lists that the original note puts on the air do not fit. They are carried later, in HELLO, once a CoC exists. A scanner that sees only the advertisement can already dial: the BD_ADDR in the value

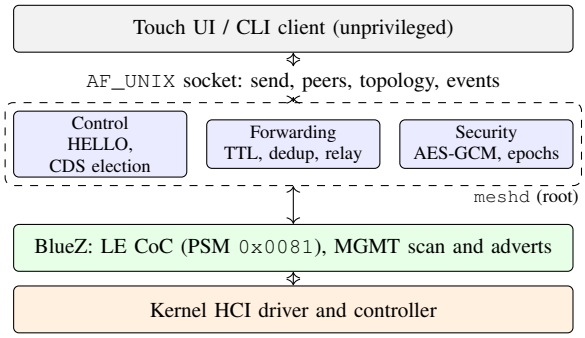


Fig. 1. One BB node. The daemon owns the adapter; displays are clients and never hold a link.

is the address that accepts PSM `0x0081`. It cannot be taken from the advertising address itself, because a non-connectable advert may rotate that address.

Joining is then four steps, and they are not the same timeout. Presence is one advertising interval. The CoC dial is one L2CAP handshake. HELLO fills the neighbour table and starts the miss timer. ANNOUNCE, on a 30 s period, is the step that delivers public keys beyond one hop and thereby closes the revocation window of Section III-E. A spectator who taps "send" during the first of these steps is early; the mesh is up, the bubble is not yet able to exclude anyone safely. The panels therefore show link state separately from group epoch.

C. Routing header and packet types

Every frame begins with a 26-byte cleartext header:

```
type(1) | src(6) | dst(6) | ttl(1) | seq(4 BE) | ts_ms(8 BE)
```

Relays decrement TTL on the wire, so TTL is *not* in the AES-GCM additional authenticated data; the rest of the header is. A relay without the group key can still forward, and cannot forge an AAD-covered field; it can rewrite TTL (Section III-F). Broadcast destination is `FF:FF:FF:FF:FF:FF`.

The ciphertext is AES-256-GCM [16], [17] with a 16-byte tag. There is no nonce on the wire. Each packet is sealed under its own key,

$$k_{\text{pkt}} = \text{HKDF}_{\text{SHA256}}\big(K;\ src \,\|\, seq \,\|\, ts_ms\big),$$

32 bytes, with K as input key material, the three fields as *info*, and no salt. Here K is the group key (or a pairwise key) and the three fields are the header's 6-byte source, 4-byte sequence and 8-byte timestamp. The GCM nonce is then fixed at zero; uniqueness rests on the key, which changes whenever (src, seq, ts_ms) does. Pairwise keys are HKDF-SHA256 over an X25519 shared secret, with the epoch in *info*. Overhead is 42 bytes. Against a conservative 512-byte MTU that leaves 470 bytes of plaintext. Both boards negotiate L2CAP MTU 672, so a maximum-size message is still one frame; 512 is policy. A 471-byte payload is refused, never truncated.

Two sequence counters number these frames (Hello/ANNOUNCE versus DATA/REKEY). They must not share an anti-replay window (Section V-D).

TABLE III
BB PACKET TYPES.

Type	Name	Plane	Role
`0x01`	HELLO	both	Neighbour table, epoch, CDS
`0x02`	DATA	private	Group broadcast
`0x04`	SUBGROUP	private	Addressed multicast
`0x05`	REKEY	private	Pairwise epoch catch-up
`0x06`	ANNOUNCE	private	Signed pubkey binding
`0x07`	REKEY_FLOOD	private	Coordinator revocation

D. CDS, forwarding, and membership expiry

Roles follow Wu–Li 1999 [4]. MPR [6] is implemented in the simulator; boards run Wu–Li. A node forwards if it is RELAY, or if it has two or more L2CAP links (a bridge the CDS has not named—the usual case at the edge of range, where the advertisement graph is a clique and the L2CAP graph is a star). Duplicate suppression is keyed by (src, seq) and by which counter produced seq.

A neighbour is dropped after `HELLO_MISSES` consecutive silent Hello periods (default 3, minimum 2), i.e. 15 s at a 5 s interval. Counting periods keeps the threshold meaningful if the interval is retuned.

E. Security and revocation

Link encryption is off. Group AEAD is AES-256-GCM under the group key; pairwise keys are X25519 [18] after ANNOUNCE; rekeys are attested with Ed25519 [19]. The routing header is visible to a sniffer; payloads are not.

A passive sniffer in range therefore learns who transmitted, toward which broadcast or unicast address, and how far the frame may still travel. It does not learn the text, the group key, or the pairwise secrets. Every header field except TTL is covered by the additional authenticated data, so changing source, destination, sequence or timestamp makes the tag fail at a member. An active attacker who injects a new frame has to produce a valid tag under a key they do not hold. TTL is excluded on purpose, so that relays can decrement it, and an attacker *can* therefore rewrite it; that limit is part of the threat model below. What this does *not* give is anonymity of the radio address: BD_ADDR is in the header and on the advertisement, on purpose, because that is how a neighbour is dialled without a bond. Anonymous identifiers hide the group membership of a presence record; they do not hide the controller.

The coordinator is the member with the smallest anon_id among *all known members* (Hello-resolved *and* ANNOUNCE-learned). Election and validation of a rekey must use that same set (Section V-D).

Only the coordinator may revoke. It bumps the epoch, rotates the group key, and floods `REKEY_FLOOD` wrapped under *pre-bump* pairwise keys, excluding the victim. Late members catch up on the next stale Hello. The coordinator cannot wrap a key it does not hold: Hello supplies direct neighbours only; everyone else arrives in ANNOUNCE (30 s). Revoking inside that *join window* is refused or strands legitimate members. This is an operating limit of the protocol.

On both boards AES-GCM runs on the ARMv8 Cryptography Extensions (`gcm-aes-ce` via OpenSSL). On the two kernels of Table II, the Qualcomm crypto engine driver (QCE) does not register a GCM implementation, so it is not on the path.

F. Threat model and accepted limits

The adversary is any radio in range, member or not. Confidentiality and integrity of payloads against non-members are the goals above. Availability and fairness among members are not guaranteed, and the current implementation accepts the following.

a) Flooding by a non-member.: The topological plane is open by design: a relay forwards frames it cannot authenticate. A non-member can therefore inject well-formed frames with invalid tags and have the CDS carry them. Because TTL is outside the AAD, it can also reset TTL on a frame in transit and extend its reach to the configured maximum. Duplicate suppression bounds each (src, seq) to one forward per relay, so neither attack loops, but both cost airtime. Relays do not rate-limit by source today.

b) Poisoning the duplicate cache.: (src, seq) is in clear and relays cannot check the tag. An attacker who emits a forged frame carrying a member's (src, seq) before the genuine one reaches a relay makes that relay drop the genuine frame. The effect is selective loss, not disclosure; members still reject the forged copy. Keying relay suppression on a tag prefix, or letting members that hold the key re-forward on a miss, are the obvious counter-measures.

c) Choosing the coordinator.: A member's anon_id is $\mathrm{HMAC\text{-}SHA256}(K_{\mathrm{boot}}, BD_ADDR)$ truncated to 64 bits. A non-member cannot compute it. A member that controls its own BD_ADDR (a random static address) can, however, try addresses until its anon_id is the smallest and so become coordinator, the only node allowed to revoke. BB trusts members not to do this; a verifiable election is future work.

d) Traceability.: As stated above, BD_ADDR is stable and in clear. A passive observer can follow a board and learn who transmits toward whom. Payload privacy does not imply location privacy.

G. One group message, hop by hop

Consider a chat line sent by screen A toward screen B, with a UNO Q relay R between them and no direct A–B link. A builds a `DATA` frame: broadcast destination, TTL set to the configured maximum, sequence taken from the data counter, timestamp in milliseconds. The payload is the UTF-8 text. The per-packet key is derived from the current group key, A's address, the sequence and the timestamp; AES-GCM seals it. The 26-byte header stays clear.

A writes that frame on every CoC it holds. R accepts it, checks the relay anti-replay window for (A, seq) in the data counter's space, decrements TTL, and writes the same ciphertext to its other peers. R does not need the group key to do this. It then tries local delivery: GCM fails if R is not a member, and the frame is still forwarded. B decrypts, checks its own delivery window, and hands the text to the local UI. A copy that returns to A is dropped because the source is local. A second copy that arrives at B through another relay hits the same (src, seq) window and is dropped.

Two consequences follow. First, adding a relay does not require that relay to hold the group key: the open mesh and the private bubble are separated on the wire, not only in the specification. Second, the UI can show the ingress neighbour and a hop count ($ttl_{\max} - ttl + 1$) without reading any node beyond its own daemon. That is what a spectator sees move on the screen.

H. Platform-specific link discipline

The two software stacks of Table II do not present the same HCI behaviour, and the mesh only stays up if the implementation respects both. The three behaviours below were observed with those exact images—on the UNO Q, Debian 13, kernel 7.0.0-g122c2c22d838, BlueZ 5.82; on the RUBIK Pi 3, Ubuntu 24.04.3, kernel 6.8.0-1055-qcom, BlueZ 5.72—and newer kernels or firmware may remove them.

a) Central-only links.: On the UNO Q image, the WCN399x controller is driven by `hci_uart_qca`. That driver does not report valid LE states, so the kernel sets `HCI_QUIRK_BROKEN_LE_STATES`. While the adapter is peripheral on any link, `check_pending_le_conn()` never sends an LE Create Connection. A dial then times out with no connection attempt on the air. The RUBIK Pi 3 image drives its Broadcom controller through `hci_uart_bcm` and does not set this quirk. Each UNO Q is therefore the central of every link it holds: it advertises non-connectable and dials, and each RUBIK advertises connectable and never dials. Two screens never link to each other, nor do two relays, so every path alternates screen and relay. With three screens and three relays the steady graph is the complete bipartite graph $K_{3,3}$.

b) Signalling-identifier leak.: Linux kernels that carry commit `6c3ea155e5ee` ("L2CAP: Fix not tracking outstanding TX ident") but not its follow-up `6e1930ece855` never release the signalling identifier of an LE Flow Control Credit, which is sent for every received PDU. The UNO Q kernel (7.0.0) is in that range; the RUBIK kernel (6.8) predates it and is unaffected. The pool is 254 identifiers per ACL. Under a quiet chat this is invisible; under the topology traffic of a five-node mesh a link exhausts the pool in minutes and the kernel logs `Unable to allocate ident`. On affected kernels the daemon recycles a link after 200 received credits: it drops the CoC and disconnects at the MGMT layer, then dials again. Neighbours treat this as an ordinary flap. The alternative, restarting BlueZ or the board, is what a day-long demonstration cannot afford.

c) Controller UART stall.: On the UNO Q image, the HCI UART to the WCN399x has stopped answering commands, with transmit timeouts and a firmware version read of -110. We saw it once on an idle bench after about six weeks of uptime, before any recovery existed, and a reboot cleared it. Under five-node mesh traffic it has recurred after a few hours. We have not established that the two occurrences

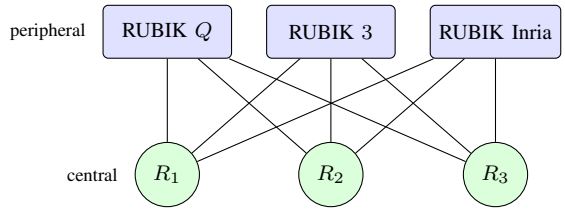


Fig. 2. Demonstration topology, $K_{3,3}$. Each UNO Q relay is central toward all three screens. A message between two screens crosses one of three two-hop paths; the receiving screen draws that path as the text arrives.

share a cause. Rebinding the HCI UART driver reloads the firmware and restores a powered adapter in tens of seconds, without rebooting the board. Each daemon watches its own controller and does this locally: a restart order flooded on the mesh cannot reach a radio that is already dead.

IV. The demonstration stand

Figure 2 is the configuration used for demonstrations. Three RUBIK Pi 3 boards carry touch panels; we call them RUBIK Q (the operator screen), RUBIK 3 and RUBIK Inria. Three Arduino UNO Q boards, R_1 to R_3, are relays and have no display. The panels show preset sentences rather than a keyboard, the live topology beside the transcript, and, on RUBIK Q only, an operator action that revokes a chosen member.

Revocation on that stand is the protocol of Section III-E, not a second mechanism. The operator names a board; the coordinator omits that board from `REKEY_FLOOD`; the next preset sentence is readable on every remaining member and not on the victim. The victim's panel says it has been excluded, which is the point of the demonstration: the mesh is still there, the bubble is not. An explicit operator reset returns every board, including the victim, to the bootstrap epoch-0 key so the same table can be shown again. That reset is a demonstration control. It is not part of the group protocol, and a revoked member cannot invoke it.

Sections V-A to V-D report controlled runs on two to six boards and in simulation, collected before this arrangement. Section V-E reports measurements on the stand itself and on the same six boards spread through a building.

V. Evaluation

Experiments use simultaneous probes on all boards, the same git commit, and *worst-board* figures (a mesh is complete when the last node has joined). Warm-up (25 s) is counted as convergence, not as instability. Departure is timed on the presence table, not on L2CAP sockets (a socket dies with its process regardless of `HELLO_MISSES`). Load tests use a single sender; each message carries an index, and delivery is that index seen by a listener. Simulation instantiates the shipped forwarder on a unit-disk radio and drains floods breadth-first (call-stack, depth-first delivery is not a radio).

TABLE IV
Mesh formation and chat delivery.

Setting	n	Full mesh	Steady complete	Delivery
Automated	3	7–14 s	97–98%	100%
Automated	2	3–14 s	97%	100%
Manual soak	6	∼40 s	stable after 60 s	427/429

TABLE V
Departure delay vs. configured Hello misses (presence view).

`HELLO_MISSES`	Expected	Measured departure	Rejoin
3	15 s	17–21 s	7–12 s
6	30 s	32–36 s	7–10 s

A. Convergence, stability, and departure

The two losses in 429 were the first message (before convergence) and the last (during shutdown); the delivery column of Table IV and the abstract refer to these runs. Applications should display mesh state rather than assume delivery in the first tens of seconds. Relay role flips in the automated runs: 0. Steady-state L2CAP flaps: a few per run, all recovered.

Killing a node without notice (Table V): departure tracks the configured miss count. Rejoin does not: it is discovery and dial.

B. Crypto is not the bottleneck

BLE CoC carries a few hundred packets per second. AES-GCM is two to three orders of magnitude above the link (Table VI). Energy of the crypto path is not measured.

C. Topological and revocation limits

Random geometric graphs are drawn in a square whose area gives a *nominal* mean degree of 6, ignoring borders; nodes near a border have fewer neighbours, so the measured mean degree is lower (3.3–5.2 in Table VII). Draws are kept only if connected. Each draw is probed from three sources spread through the node set. On every draw, and on chains, the minimum TTL that delivers to every node equals the largest hop eccentricity among those sources:

There is no single node count at which the mesh breaks. Once the eccentricity exceeds 10 hops, default TTL 10 no longer spans the network—about twenty nodes at this density. At six nodes the default is ample. Density has its own floor: at $n = 50$, no placement out of 100 was connected at a nominal degree of 3 or 4, while nominal degree 5 gave connected draws with a measured mean degree of 4.6 (percolation). Denser graphs elect fewer relays (52%→38%) and cost more per flood.

After ANNOUNCE has run, revocation at $n = 6$–30: 0 stranded members, 0 leaks to the victim. Cost grows from 12 to 83 emissions per node, that is roughly linearly in n per node (about $2n$ to $3n$), so the total cost of one revocation grows roughly quadratically. Before ANNOUNCE, revocation

TABLE VI
AES-256-GCM ON A 470-BYTE PAYLOAD (MEAN OF 20 000 CALLS).

Platform	Encrypt + decrypt	Equivalent rate
RUBIK Pi 3	$3.0 + 2.8\,\mu$s	173 000 pkt/s
Arduino UNO Q	$18.2 + 15.2\,\mu$s	30 000 pkt/s

TABLE VII
MINIMUM TTL FOR FULL DELIVERY EQUALS THE WORST SOURCE'S HOP ECCENTRICITY (WU–LI, SHIPPED FORWARDER, GEOMETRIC RADIO, NOMINAL DEGREE 6, MEASURED DEGREE SHOWN).

n	Degree	Ecc. (hops)	Min TTL	Deliv. @ TTL 10
10	3.9	4	4	100%
20	4.3	8	8	100%
30	3.3	11	11	93%
50	5.2	13	13	94%
100	5.2	19	19	23%

is often refused. Operating rule: wait one ANNOUNCE period after a join before revoking.

D. Load, and the two implementation defects

This load study ran on the images of Table II. A single sender, 64–470 byte payloads. The RUBIK listener received 100% through 30 msg/s. The UNO Q tracked to 10 msg/s, lost at 15, then delivered 0—including a 1 msg/s recovery step after 25 s idle—while its links stayed nominally up. The kernel logged `Unable to allocate ident: -28` (`ENOSPC` on BlueZ L2CAP *signalling* identifiers); controller error counters stayed 0. Data-plane saturation delays Hellos, neighbours expire, auto-dial bursts, the ident pool empties. This is the kernel leak of Section III-H; the run predates link recycling. Restarting the Bluetooth service recovers the board. The controller UART stall of the same section is a different failure: the controller stops answering, and a service restart does not help.

The limit studies also exposed the two implementation defects announced in Section I. Both are fixed in the code evaluated here, and each fix carries a regression test.

a) Defect 1: shared anti-replay window.: Hello/ANNOUNCE and DATA were numbered from independent counters but checked against one window. A DATA frame whose sequence equalled an earlier ANNOUNCE from the same node was dropped by every receiver and every relay. Reproduction: sequence 4 delivered 5/5, sequence 5 (the ANNOUNCE) 0/5, sequence 6 delivered 5/5. Fix: each counter has its own window; the test checks that an ANNOUNCE no longer masks a DATA frame with the same sequence.

b) Defect 2: split-brain coordinator.: Election used Hello-resolved neighbours only; validation of a rekey also counted ANNOUNCE-learned peers. On a stringy 30-node graph (mean degree 3.3) four nodes declared themselves coordinator; revocation was refused mesh-wide while the initiator's epoch advanced; 28 of 29 legitimate members stayed on the old key. Fix: election and validation now use one membership view (Hello-resolved and ANNOUNCE-learned members, Section III-E); the test checks that a sparse mesh elects exactly one coordinator.

TABLE VIII
FOUR-HOP LINE, 8 MIN 45 S, 79 GROUP MESSAGES ACCEPTED BY THE SENDER. DELAY IS SEND-TO-DISPLAY, CLOCK-CORRECTED.

Board	Hops	Received	Median	90th pct.
R_3	1	—	85 ms	103 ms
RUBIK 3	2	79/79	101 ms	176 ms
R_1	3	79/79	143 ms	213 ms
RUBIK Inria	4	76/79	162 ms	242 ms

E. What the six-node stand measured

The following runs were collected on the stand of Figure 2. Each of the three screens held three links and each of the three relays held three: the nine edges of $K_{3,3}$.

Fifteen group messages, sent from one screen a few seconds apart, were received by all five other boards: 75 receptions out of 75. Round to round, the arrival time on a given board moved by less than a tenth of a second. Timed over SSH, that run cannot resolve a one-way delay in milliseconds; the four-hop run below, logged on the boards themselves, does.

Revocation matches Section III-E on this hardware. After one screen was excluded it did not display the next message; the other four boards did. Resetting the demonstration put all five back on the bootstrap key, and the next message reached all five. Excluding one relay left both screens able to read the message. The excluded relay did not.

Adding a node is dominated by the boot, not by the mesh. Rebooting the third screen, the first group message it could decrypt arrived 142 s after the reboot command and 4.4 s after its mesh daemon was running. Cutting a UNO Q controller loose on purpose, that relay received a group message again 130 s later, without a reboot of the board.

a) A four-hop line.: Every board so far sat in one room, so a two-hop path was a choice of role, not evidence of range. We then ran the boards on battery, spread them through the building with one UNO Q switched off, and let the mesh form on its own. It settled into a single line, RUBIK $Q \to R_3 \to$ RUBIK 3 $\to R_1 \to$ RUBIK Inria, in which each board held links only to its neighbours. Screens and relays alternate because two screens never dial each other and two UNO Qs cannot (Section III-H). Boards were about 15 m apart, so the line spanned about 60 m indoors.

Without Wi-Fi at the far positions, each board logged every received message and every link change to local storage, and RUBIK Q sent a numbered group message every 5 s. Board clocks agreed within 50 ms before and after the run and drifted by under 10 ms, so one-way delay is resolved to a few tens of milliseconds (Table VIII).

The three messages RUBIK Inria missed were the first three, sent while the line was still forming. R_3's own log was cut short (below), so its row gives delay only; the two boards behind it received every message, which bounds its losses at

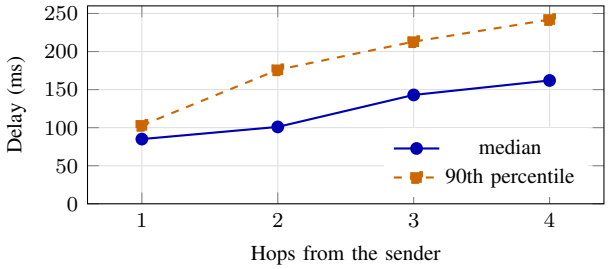


Fig. 3. Send-to-display delay against hop count on the four-hop line (Table VIII).

zero for that window. Each hop beyond the first adds a few tens of milliseconds (Figure 3).

Three events happened during the run. A mesh restart ordered from RUBIK Q had four-hop delivery back about 50 s later. R_3, the sender's only neighbour, lost power with no shutdown in its journal; the whole line was dark for about 95 s, until it had rebooted and redialled. A single relay is a single point of failure, which is why the stand normally runs three. Finally, RUBIK Inria was carried away from R_1. Its link held to about 30 m and then dropped. When the board was brought back, the link returned within seconds of it coming into range, and delivery resumed at four hops.

What remains open is a day of unattended delivery, and the maximum spacing of each hop: only the last one was pushed until it broke.

VI. Conclusion

An open BLE mesh with private groups is implementable without pairing. On the boards we have, the mesh converges in seconds, the CDS does not stretch paths, and the default TTL is a hop budget rather than a magic node count. Revocation works once keys have been announced and fails in a documented window if they have not. Cryptography is not the throughput limit; the BlueZ stack of the UNO Q, on the kernel we tested, is. Keeping that stack up for a day further requires the link discipline of Section III-H: the UNO Q must be central, its CoCs must be recycled before the signalling-identifier pool runs out, and a dead controller must be recovered by the board that holds it.

Section V-E is the first timed pass on the six-board stand of Section IV: delivery, revocation, boot-to-group, recovery from a dead controller, and a four-hop line of about 60 m through the building at 162 ms median delay, whose last link held to about 30 m. A full day of delivery, and the breaking distance of every hop, remain open. Energy of AES-GCM versus the radio, and an Android CoC port, are future work.

The link-change logs every board already keeps form a timestamped contact graph. The unit-disk simulations of Section V could be replayed on real mobility traces instead of static placements, which would tell how CDS election and revocation behave when people, not boards, move.

Acknowledgment

The RUBIK Pi 3 and Arduino UNO Q boards were provided through the Qualcomm University Platforms program. The protocol note [2] acknowledges Matthieu Finiasz (OLVID) and Thomas Jaussouin and Olivier Berni (Lunabee); that debt carries over. AI-assisted tools were used for software development, data analysis and editing of this manuscript; the authors checked all results and are responsible for the content.

References

[1] *Mesh Profile Specification, Version 1.0*, Bluetooth SIG, 2017.
[2] N. Achir and P. Jacquet, "BUBBLE-BLUE a multihop private network based on Bluetooth," *arXiv preprint arXiv:2509.00967*, 2025.
[3] *Bluetooth Core Specification, Version 5.0*, Bluetooth SIG, 2016, vol. 3, Part A: LE L2CAP Connection-Oriented Channels; v5.1 adopted in 2019.
[4] J. Wu and H. Li, "On calculating connected dominating set for efficient routing in ad hoc wireless networks," in *Proc. 3rd Int. Workshop on Discrete Algorithms and Methods for Mobile Computing and Communications (DIAL-M)*, 1999, pp. 7–14.
[5] ——, "A dominating-set-based routing scheme in ad hoc wireless networks," *Telecommunication Systems*, vol. 18, no. 1, pp. 13–36, 2001.
[6] A. Qayyum, L. Viennot, and A. Laouiti, "Multipoint relaying for flooding broadcast messages in mobile wireless networks," in *Proc. 35th Hawaii Int. Conf. on System Sciences (HICSS)*. IEEE, 2002, pp. 3866–3875.
[7] T. Clausen and P. Jacquet, "Optimized link state routing protocol (OLSR)," IETF, RFC 3626, 2003.
[8] P. Jacquet, "Analytical results on connected dominating sets in mobile ad hoc networks," INRIA, Tech. Rep., 2004.
[9] K. Kanchanasut, T. Wongsaardsakul, M. Chansutthirangkool, A. Laouiti, H. Tazaki, and K. R. Arefin, "DUMBO II: a V-2-I emergency network," in *Proc. 4th Asian Conf. on Internet Engineering*, 2008, pp. 37–38.
[10] J. Dorsey *et al.*, "BitChat: Bluetooth mesh chat," https://github.com/permissionlesstech/bitchat, 2025.
[11] The Briar Project, "Briar," https://briarproject.org/, 2024.
[12] M. Qvist, "Reticulum / LXMF," https://github.com/markqvist/Reticulum, 2024.
[13] Bridgefy Inc., "Bridgefy," https://bridgefy.me/, 2024.
[14] M. R. Albrecht, J. Blasco, R. B. Jensen, and L. Marečková, "Mesh messaging in large-scale protests: Breaking Bridgefy," in *Topics in Cryptology – CT-RSA 2021*, ser. LNCS, vol. 12704. Springer, 2021, pp. 375–398.
[15] C. Ballinger, "BLEMeshChat: Bluetooth LE mesh chat for iOS and Android," https://github.com/chrisballinger/BLEMeshChat, 2014.
[16] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST, NIST SP 800-38D, 2007.
[17] J. Daemen and V. Rijmen, *The Design of Rijndael: AES — The Advanced Encryption Standard*. Springer, 2002.
[18] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," IETF, RFC 7748, 2016.
[19] S. Josefsson and I. Liusvaara, "Edwards-Curve digital signature algorithm (EdDSA)," IETF, RFC 8032, 2017.